

VeriSilicon AHB DMAC Specification

(VS_AHBDMAC)

REV: 1.7

2024/02/28

VeriSilicon Microelectronics (Shanghai) Co., Ltd.

Checked by: _____

Approved by: _____

Trademark Acknowledgments

VeriSilicon & the VeriSilicon logo are the trademarks of VeriSilicon Microelectronics (Shanghai) Co., Ltd.

All other products and company names mentioned may be the trademarks of their respective owners.

© 2014 VeriSilicon Microelectronics (Shanghai) Co., Ltd. All rights reserved.

Printed in P.R.China.

VeriSilicon Microelectronics (Shanghai) Co., Ltd. reserves all its copy rights and other intellectual property rights, ownership, powers, benefits and rights arising or to arise from this manual. All or part of the contents of this manual may be changed by VeriSilicon Microelectronics (Shanghai) Co., Ltd. without notice at any time for any reason, including but not limited to improvement of the product relating hereto.

VeriSilicon Microelectronics (Shanghai) Co., Ltd. shall not undertake or assume any obligation, responsibility or liability arising out of or in respect of the application or use of the product described herein, except for reasonable, careful and normal uses.

Nothing, whether in whole or in part, within this manual can be reproduced, duplicated, copied, changed or disposed of in any form or by any means without prior written consent by VeriSilicon Microelectronics (Shanghai) Co., Ltd..

VeriSilicon Microelectronics (Shanghai) Co., Ltd.

20A, No.560, Songtao Road, Zhangjiang Center, Zhangjiang Hi-Tech Park,
Pudong New Area, Shanghai 201203,, P. R. China

Tel : +86-21-5131-1118

Fax : +86-21-5131-1119

Web : <http://www.verisilicon.com>

[illegible]

Contents

1.	General Description.....	6
2.	Feature List	6
3.	Application System Circuit.....	6
4.	Block Diagram	6
5.	Pin List and Assignment	7
6.	Block Description.....	9
6.1.	APB Slave Interface	9
6.2.	DMAC Hardware Handshake Interface	9
6.3.	Interrupt.....	12
6.4.	AHB Master Interface	13
6.4.1.	Early Termination of DMAC Channel.....	13
6.4.2.	Direct DMA Mode	14
6.4.3.	Ring buffer descriptor Mode	14
6.5.	Priority Arbiter.....	16
6.6.	Channel FIFO.....	16
7.	Register File and Memory Mapping Description.....	17
7.1.	Register and Memory Mapping Summary Table.....	17
7.2.	Channel Register Description.....	19
7.2.1.	CHNLn_SRC_ADDR_L.....	19
7.2.2.	CHNLn_SRC_ADDR_H.....	19
7.2.3.	CHNLn_DST_ADDR_L.....	20
7.2.4.	CHNLn_DST_ADDR_H.....	20
7.2.5.	CHNLn_CONFIG	20
7.2.6.	CHNLn_TRANS_SIZE	22
7.2.7.	CHNLn_QUEUE_ADDR_L.....	22
7.2.8.	CHNLn_QUEUE_ADDR_H.....	22
7.2.9.	CHNLn_QUEUE_LEN.....	22
7.2.10.	CHNLn_QUEUE_IDX.....	23
7.2.11.	CHNLn_CONTROL	23
7.2.12.	CHNLn_INT_MASK	25
7.2.13.	CHNLn_INT_STATUS	25
7.2.14.	CHNLn_STATUS.....	26
7.3.	Control Register Description.....	27
7.3.1.	DMAC_RST	27
7.3.2.	DMAC_INT_STATUS	27
7.3.3.	DMAC_INTMSK	28
7.3.4.	DMAC_CAP	28
8.	Interface Timing Diagram	29
8.1.	Peripheral Handshaking Interface	29

8.2.	APB Bus Interface	30
8.3.	AHB Bus Interface	30
9.	Configuration Program Flow.....	30
10.	Application Note	31
10.1.	Gate Count	31
10.2.	Channel Number Definition	31
10.3.	System Considerations.....	31
10.4.	Other Information	31
11.	Absolute MAHBmum Ratings	31
12.	Recommended Operation Conditions	32
13.	Electrical Characteristics	32

1. General Description

The VS_AHBDMAC core is compliant with the AHB-LITE and APB3 Specification. It can be programmed over the APB3 slave interface with DMA data transferring via AHB bus master interface. This design provides configurable channel number which can be up to 16 DMA channels.

Each DMA channel supports four types of transfers: Memory-to-memory, memory-to-peripheral, peripheral-to-memory and peripheral-to-peripheral. Each DMA channel is assigned a fixed priority.

2. Feature List

- AHB-LITE master and APB3 slave, slave address/data width is 32, and master data width configurable between 32/64, master address width configurable.
- 1~16 configurable channels, each channel can support unidirectional transfer.
- 1~16 groups configurable DMA hardware handshake signals, providing 1~16 groups peripheral DMA request lines, support asynchronous handshake signals.
- Fixed DMA channel priority, channel 0 has highest,..., channel 15 has lowest priority.
- Supports Memory-to-memory, memory-to-peripheral, and peripheral-to-memory transfers, peripheral-to-peripheral.
- Support single,incr4/8/16 burst type for source and destination.
- Support ring buffer descriptor mode.

3. Application System Circuit

The following Figure 3-1 shows the system circuit structure

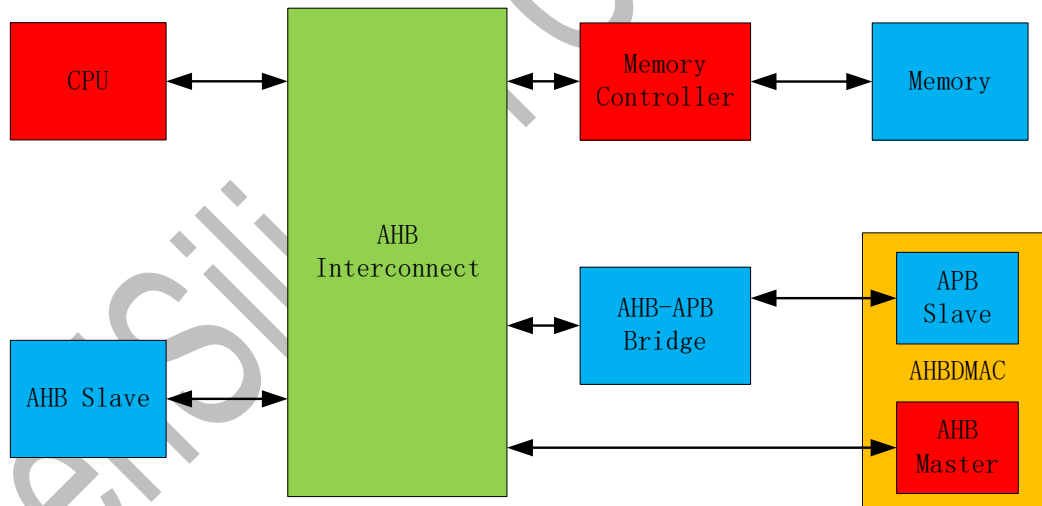


Fig. 3-1 Scheme of Application System Circuit

4. Block Diagram

The VS_AHBDMAC core supports four types of transfers:

- Memory to memory
- Memory to peripheral
- Peripheral to memory
- Peripheral to peripheral

The following Figure 4-1 shows the VS_AHBDMAC block diagram.

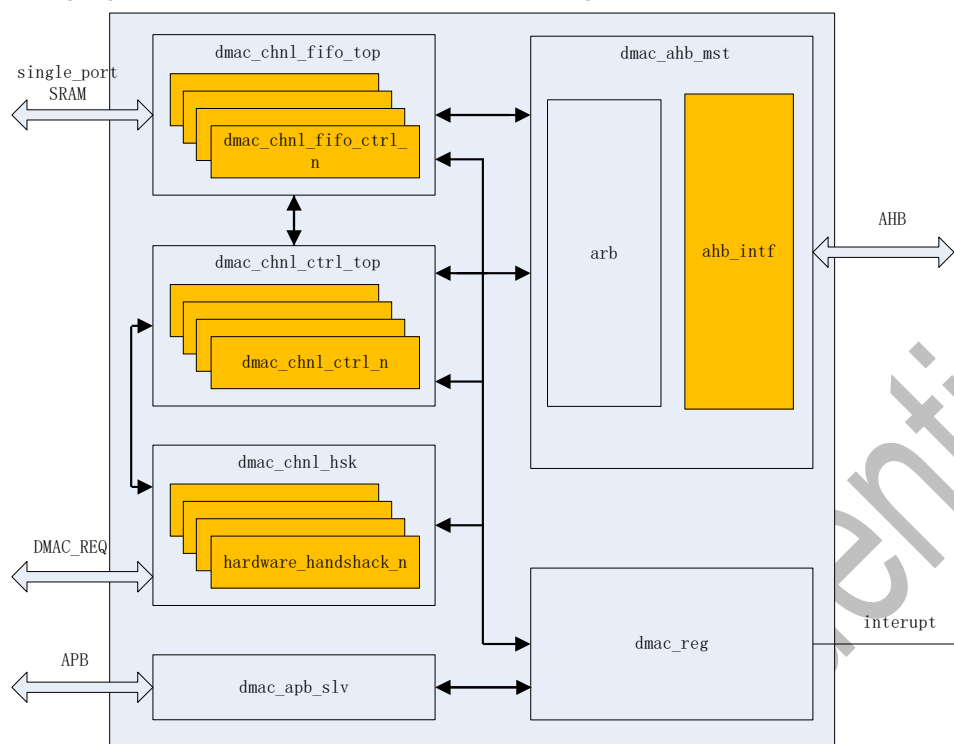


Fig. 4-1 The VS_AHBDMAC Block Diagram

5. Pin List and Assignment

Signal Name	MI/O	Description
Parameter		
ADDR_WIDTH	NA	AHB master address width
DATA_WIDTH	NA	AHB master data width, 32bit or 64bit
CHN_NUM	NA	DMA channel number, from 1 to 16
HS_NUM	NA	DMA handshake number, from 1 to 16
Clock and Reset Signals		
pclk	I	APB clock signal, which is synchronized with hclk and can be of different frequencies.
hclk	I	AHB clock signal, which is synchronized with pclk and can be of different frequencies.
presetn	I	pclk reset signal, low active
hresetn	I	hclk reset signal, low active
APB BUS Signals		
paddr[31:0]	I	APB bus signals For more details, please refer to the APB4 specification.
psel	I	
penable	I	
pwrite	I	
pwdata[31:0]	I	
pready	O	

pslverr	O	
prdata[31:0]	O	
AHB BUS Signals		
haddr[ADDR_WIDTH-1:0]	O	For more details, please refer to the AMBA3 AHB Specification.
hwrite	O	
hsize[2:0]	O	
hburst[2:0]	O	
hprot[3:0]	O	
htrans[1:0]	O	
hmastlock	O	
hready	O	
hwdata[DATA_WIDTH-1:0]	O	
hreadyout	I	
hresp	I	
hrdata[DATA_WIDTH-1:0]	I	
Peripheral Handshake Signals		
dma_req[HS_NUM-1:0]	I	For more details, please refer to the 6.2.
dma_trx_single[HS_NUM-1:0]	I	
dma_last[HS_NUM-1:0]	I	
dma_ack[HS_NUM-1:0]	O	
dma_complete[HS_NUM-1:0]	O	
Single-port SRAM		
sp_clk	O	Single-port SRAM clock, it is AHB clock
sp_wen	O	Single-port SRAM write enable, active high
sp_cen	O	Single-port SRAM chip enable, active high
sp_addr[SP_ADDR_WIDTH-1:0]	O	Single-port SRAM address width, SP_ADDR_WIDTH depends on the CHNL_NUM, please refer to the section 14.2
sp_wdata[DATA_WIDTH-1:0]	O	Single-port SRAM write data, the data width is same as AHB bus data width
sp_rdata[DATA_WIDTH-1:0]	I	Single-port SRAM read data, the data width is same as AHB bus data width
Interrupt Signals		
dmac_int	O	Combined all interrupts, high active

6. Block Description

6.1. APB Slave Interface

All transactions on the APB slave programming bus of the VS_AHBDMAC are used to access all DMAC registers.

6.2. DMAC Hardware Handshake Interface

There are five types of signals in the DMAC hardware handshake interface, which are `dma_req`, `dma_trx_single`, `dma_last`, `dma_ack`, and `dma_complete`. The first three are inputs and the last two are outputs. This interface also supports asynchronous handshake signals.

The meaning of the handshake signals are as follows:

- DMAC control flow: At this time DMAC is the flow controller, and `dma_req` and `dma_trx_single` signals are sampled by DMAC.
 - `dma_req`: If the peripheral is the source device, the `dma_req` signal represents the amount of data available in the source peripheral, which is greater than or equal to `CHNL_CONFIG.SRC_WIDTH*CHNL_CONFIG.SRC_BST_LEN`. If the peripheral is the destination device, then the `dma_req` signal represents that the data storage space that can be used in the destination peripheral is greater than or equal to `CHNL_CONFIG.DST_WIDTH*CHNL_CONFIG.DST_BST_LEN`. Note that if the address is unaligned, even if the handshake signal `dma_req` is asserted, the DMAC will actually perform an address-aligned single transaction first. When this type of transaction is completed, it will return the `dma_ack` signal. If the address is aligned, the DMAC will perform a burst transaction which burst length is equal to `SRC/DST_BST_LEN` and burst size is equal to `SRC/DST_WIDTH`, so the peripheral needs to confirm that it was actually read or write how much data, and then continue to send the corresponding handshake signals, after the `dma_ack` is asserted, `dma_req` is de-asserted after one peripheral clock cycle. Note that when a source or destination peripheral is in the single transaction region, the `dma_req` signal can still be asserted. In this case, the burst transaction is started and “early-terminated” at block completion without transferring the programmed amount of data. Only the amount of data required to complete the block transfer is transferred.
 - `dma_trx_single`: The `dma_trx_single` signal is a status signal that is asserted by a source or destination peripheral when it can transmit or accept at least one source or destination data item, otherwise it is cleared, the number of single data item is `SRC_WIDTH/8` or `DST_WIDTH/8`, unit is byte. This signal is sampled by the DMAC only in the single transaction region of the block transfer. Outside this region, `dma_trx_single` is ignored and DMAC waits for `dma_req` to be asserted. Once `dma_trx_single` is asserted, it must remain asserted until `dma_ack` is asserted. When the peripheral that is driving `dma_trx_single` determines that `dma_ack` is asserted, it must de-assert `dma_trx_single`.
 - `dma_ack`: The `dma_ack` signal is asserted after the last AHB data transfer in the current transaction (single or burst) to the peripheral has completed. It forms a handshaking loop with `dma_req` and remains asserted until the peripheral de-asserts `dma_req`. After the peripheral de-asserts `dma_req`, `dma_ack` is de-asserted after one AHB clock cycle. Note that if the address is unaligned, even if the handshake signal `dma_req` is asserted, the DMAC will actually perform an address-aligned single transaction first. When this type of transaction is

completed, it will return the dma_ack signal. At this time, the peripheral needs to confirm that it was actually read or write how much data, and then continue to send the corresponding handshake signals.

- dma_complete: This is the block transfer complete indication signal output to the peripheral. The dma_complete signal is asserted to signal block completion. This uses the same timing as dma_ack.
- Peripheral control flow(Hardware flow): At this time peripheral is the flow controller, and dma_req, dma_trx_single, dma_last are all sampled by DMAC.

NOTE: In hardware flow, the burst size(cfg_width * cfg_len) must align the cfg_addr.Example: cfg_width = 3(8byte), cfg_len = 7, cfg_addr[5:0] = 0.

- dma_req: If the peripheral is the source device, the dma_req signal represents the amount of data available in the source peripheral, which is greater than or equal to CHNL_CONFIG.SRC_WIDTH*CHNL_CONFIG.SRC_BST_LEN. If the peripheral is the destination device, then the dma_req signal represents that the data storage space that can be used in the destination peripheral is greater than or equal to CHNL_CONFIG.DST_WIDTH*CHNL_CONFIG.DST_BST_LEN. Note that if the address is unaligned, even if the handshake signal dma_req is asserted, the DMAC will actually perform an address-aligned single transaction first. When this type of transaction is completed, it will return the dma_ack signal. if the address is aligned, the DMAC will perform a burst transaction which burst length is equal to SRC/DST_BST_LEN and burst size is equal to SRC/DST_WIDTH, so the peripheral needs to confirm that it was actually read or write how much data, and then continue to send the corresponding handshake signals, after the dma_ack is asserted, dma_req is de-asserted after one peripheral clock cycle.
- dma_trx_single: When this signal is asserted by peripheral, the DMAC will perform a single transaction which size is equal to CHNL_CONFIG.SRC/DST_WIDTH. For example, if the last remain data is less than burst length, then dma_req and dma_trx_single are asserted together. The above is the case of address alignment. If the address is unaligned, even if the handshake signals dma_req and/or dma_trx_single are asserted, the DMAC will actually perform an address-aligned single transaction first. When this type of transaction is completed, it will return the dma_ack signal, so the peripheral needs to confirm that it was actually read or write how much data, and then continue to send the corresponding handshake signals. Note that dma_req is equal to dma_trx_single if CHNL_CONFIG.SRC/DST_BRT_LEN=0. Once dma_trx_single is asserted, it must remain asserted until dma_ack is asserted. When the peripheral that is driving dma_trx_single determines that dma_ack is asserted, it must de-assert dma_trx_single.
- dma_last: This signal represents whether the transaction is the last one, 1 represents the transaction is the last one, 0 represents the transaction is not the last one. If the source peripheral is a flow controller, when it asserts the dma_last signal, the DMAC will transfer the remaining data in the FIFO to the destination before it stops. If the destination peripheral is a flow controller, when it asserts the dma_last signal, the DMAC will stop after completing the last transaction, and clear the FIFO by hardware automatically.
- dma_ack: The dma_ack signal is asserted after the last AHB data transfer in the current transaction (single or burst) to the peripheral has completed. It forms a handshaking loop with

dma_req and remains asserted until the peripheral de-asserts dma_req. After the peripheral de-assert dma_req, dma_ack is de-asserted after one AHB clock cycle.

- dma_complete: This is the last burst/single transaction complete indication signal output to the peripheral. The dma_complete signal is asserted to signal block completion. This uses the same timing as dma_ack.

The following table shows all possible combinations of flow controller and hardware handshake interface signals values when the hardware handshake interface is enable.

DMAC or Peripheral Control Flow	Peripheral in Single Transaction Region?	dma_ req	dma_ trx_ single	dma_ last	VS_AHBDMAC behavior (Suppose the address is aligned)
DMAC	No	0	0	x	Not a valid transaction request.
		0	1	x	Not a valid transaction request. This signal is sampled by the DMAC only in the single transaction region of the block transfer. Otherwise, this is not a valid transaction request.
		1	x	x	Burst transaction request. dma_trx_single is not sampled by DMAC outside single transaction region.
	Yes	0	0	x	Not a valid transaction request.
		0	1	x	Single Transaction request.
		1	0	x	Burst Transaction request. DMAC initiates AHB burst of burst length needed to complete the transaction (AHB burst length < SRC/DST_BST_LEN). This is called Early Burst Termination.
		1	1	x	Burst Transaction request. DMAC initiates AHB burst of burst length needed to complete the transaction (AHB burst length < SRC/DST_BST_LEN). This is called Early Burst Termination.
		1	1	1	Last single transaction request.
Peripheral	Single Transaction Region is not applicable to Flow Control Peripheral	0	0	x	Not a valid transaction request.
		0	1	x	Not a valid transaction request. If peripheral is flow controller, DMAC waits till dma_req is asserted. The type of transaction - single or burst - is qualified by dma_trx_single input.
		1	0	0	Burst transaction request (not the last transaction).
		1	0	1	Last burst transaction request.
		1	1	0	Single transaction request (not the last transaction).
		1	1	1	Last single transaction request.

Note: 1.For peripheral-to-peripheral transfer, two peripherals cannot be flow controller at the same time. 2.Once a

peripheral asserts dma_req, dma_trx_single, dma_last signals, de-asserting these signals before asserts dma_ack is not allowed. 3.If the peripheral is the flow controller, the peripheral handshake interface must be enabled.

The DMA channel is configurable to support req/ack from peripherals. Some peripheral need handshake working with DMAC. For memory-to-peripheral, if the peripheral is ready to accept, it asserts dma_req to DMAC, then DMAC send burst data to peripheral and assert dma_ack. After peripheral de-assert dma_req, dma_ack is de-asserted. And for peripheral-to-memory, if the peripheral is ready to transmit, it asserts dma_req to DMAC, then DMAC send read command to the peripheral. After the data is read from peripheral, dma_ack asserted by DMAC. After peripheral de-assert dma_req, dma_ack is de-asserted after one AHB clock cycle.

dma_ack is asserted after the current transaction is completed, no matter single or burst transaction. If dma_last is asserted, means the peripheral control flow is completed, the current QD is expired even if the remain data length is not 0.

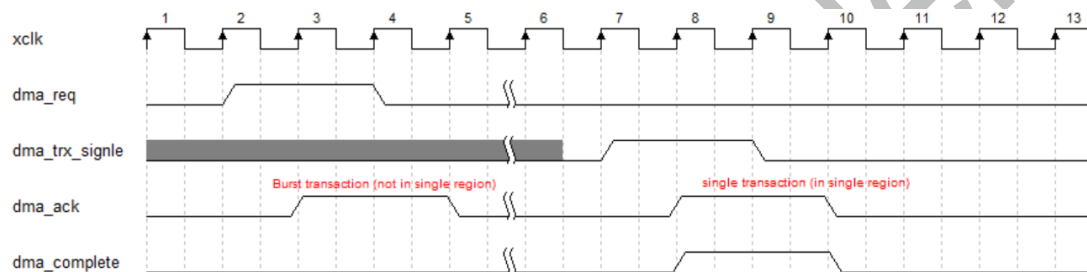


Fig. 6-1 Software flow

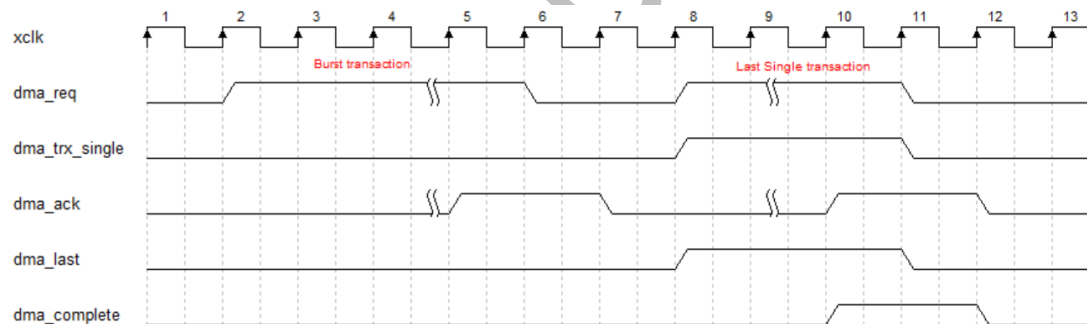


Fig. 6-2 Hardware flow

6.3. Interrupt

The interrupts can be masked by programming the relevant bits of relevant interrupt mask registers. This VS_AHBDMAC provides a single ORed interrupt of all channels to CPU.

All channels have the same structure.

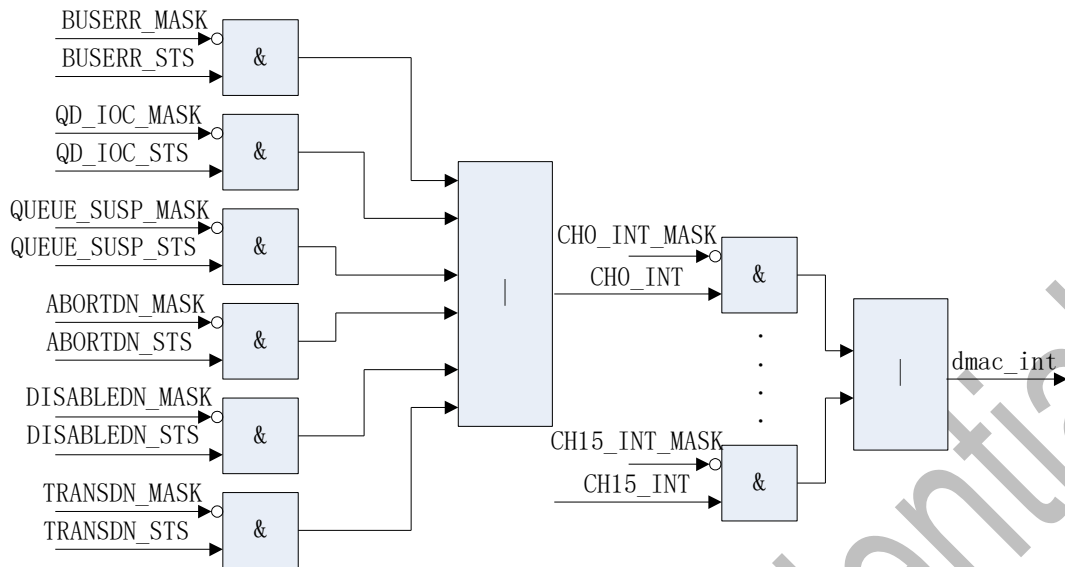


Fig. 6-3 Interrupt structure

NOTE: When the interrupt high, the software must sample CHNL_EN to confirm the DMAC working now. The reason is that when the unexpected interrupts generated by DMAC, the dmac will auto-update the CHNL_EN to 1'b0 for protect internal FSM.

6.4. AHB Master Interface

The AHB master interface is AHB-Lite master. For the last write burst of transfer, non-bufferable should be used to make sure the whole transfer data is written to destination.

6.4.1. Early Termination of DMAC Channel

During channel transferring, if register bit CH_EN is cleared by software, the last burst transfer will still be completed on the AHB master interface until the internal FSM is idle. When the CHNL disabled interrupt high, the CHNL CH_EN cleared is effective and Software should sample channel Enable status to make sure whether it could re-start DMA and channel. if a channel is re-started when the channel FIFO is not empty, the DMA will continue to complete the remain transfer size.

Another case is when channel transferring and software want stop the current block size to into another block size. Software can assert CH_ABORT to "1", and the corresponding channel will be disabled and reset after the last burst transfer be completed on the AHB master interface. And when CH_ABORT interrupt high, the software can programs the register and re-start DMA.

Interrupt is generated in each of these two early termination cases.

Below is important and key to early termination of DMA or Channel:

- Write CHNL_CTL.CH_EN to "1" will re-start channel ctrl FSM, but NOT interrupt status.
- Write CHNL_CTL.CH_ABORT to "1" will reset channel ctrl FSM, AHBM, priority arbiter and channel FIFOs except the dmac_reg.
- Write DMAC_CTL.SOFT_RST to "1" will reset channel ctrl FSM, AHBM, priority arbiter and all channel FIFOs except the dmac_reg. It will active immediately, and increase the bus hang risk

6.4.2. Unaligned transfer support

The VS_DMAL support unaligned transfer. Thus, SW can program the SRC/DST address no matter it

can be unaligned or aligned. The DMA will first to handle data considering the unaligned offset at SRC/DST.

Example:

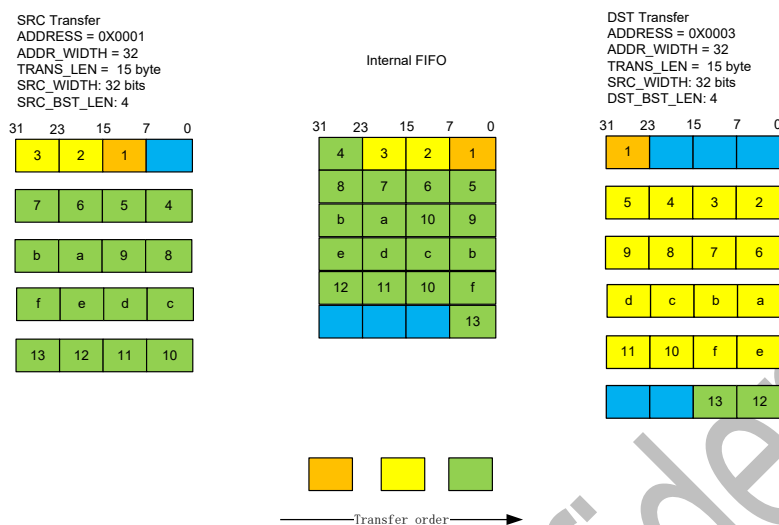


Fig. 6-4 Unaligned DMA Transfer Handling

6.4.3. Direct DMA Mode

User can select one of Direct DMA Mode and Ring buffer descriptor Mode. In direct DMA mode, ring buffer descriptor mode is ignored and vice versa. In direct mode, software configure the source and destination address, transfer data length, then start the DMA transfer. Once the data transfer is completed, an interrupt is generated.

6.4.4. Ring buffer descriptor Mode

For software performance concerning, ring buffer descriptor mode is introduced. In ring buffer descriptor mode, Software initialize the QD(queue descriptor), which describes the source/destination information. Before start DMA transaction, software allocates memory for queue ring buffer, in which the QD exists.

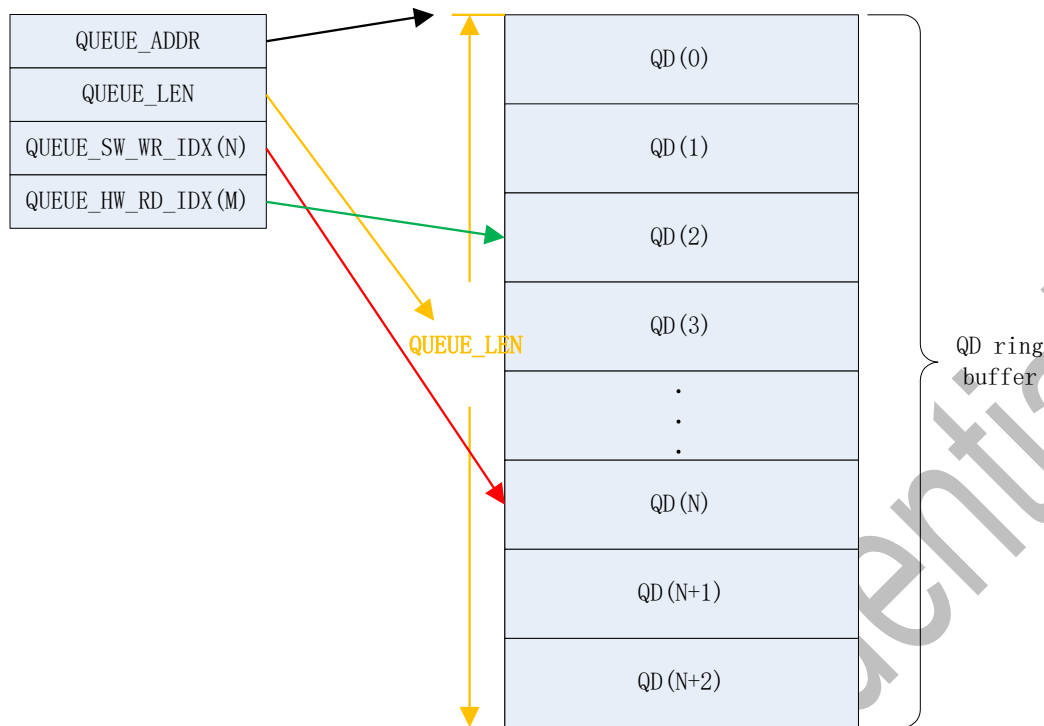


Fig. 6-5 Queue ring buffer

The queue ring buffer is a piece of system memory, where QD exist. The QD is 16 bytes or 32 bytes data structure, depending on the address width. The size of ring buffer is recommended to allocate to $2 \times N$ QD. Four additional registers are needed to support the queue ring buffer. QUEUE_ADDR points to the base address of the queue ring buffer. QUEUE_LEN defines the length of queue ring buffer with byte. QUEUE_ADDR should be aligned to QUEUE_LEN. QUEUE_SW_WR_IDX is the index of QD which the software is writing. QUEUE_HW_RD_IDX is the index of QD which the hardware is reading. The ring buffer design is similar with FIFO. When software initialize the ring buffer, it must check QUEUE_SW_WR_IDX and QUEUE_HW_RD_IDX to determine how much buffer size is available. After location (N+2) is filled, software will rollback to location 0 if the buffer is not full. If the ring buffer is empty, interrupt is generated to inform CPU filling the buffer. The data structure of QD is as follow. Since the data buffer is determined by source/destination data buffer, scattered memory management is supported.

QD(Queue Descriptor)	Bit	31	30	29:5				4	3	2	1	0
	DW7	Reserved										
	DW6	Reserved										
	DW5	Source data buffer[63:32]										
	DW4	Destination data buffer[63:32]										
	DW3	Transfer data length										
	DW2	Source data buffer[31:0]										
	DW1	Destination data buffer[31:0]										
	DW0	Reserved								IOC	SKP	RSV
	SKP: skip. 1'b1: This descriptor is skipped by hardware. It is reserved if it is useless.											
IOC: Interrupt On Completion. 1'b1 means interrupt is triggered after this descriptor is completed. It is reserved if it is useless.												

	Destination data buffer: The physical address of the destination data buffer.
	Source data buffer: The physical address of the source data buffer.
	Transfer data length: The data length to be transmitted.
	Note 0: If the address width is less than (or equal) 32, the QD is 16 bytes format, DW4~DW7 can be the next QD, with the similar field as DW0~DW3. If the address width is great than 32, then the QD is 32 bytes format, the field DW0 [31:16] is reserved.
	Note 1: If the data width is 256, the QD is only 32 bytes format no matter what the address width is.

Fig. 6-6 QD structure

6.5. Priority Arbiter

Each DMA channel has a fixed priority. Priority 0 has the highest priority and priority 15 has the lowest priority.

The priority arbiter handles in the following sequence:

1. Pick the highest priority master, and prompt its burst request to AHB channels, while holding other channels' request.
2. After current transfer is done, repeat Step1.

Considering there's no handshaking for memory-to-memory transfer, it's recommended that memory-to-memory transfers use one of the lowest priority channels. Thus this type of transaction could take use of the intervals of higher priority channels during handshaking, while not holding lower priority channels too long.

6.6. Channel FIFO

There are two internal FIFOs for each channel, one is for QD, and the other is for data. The each FIFO size is fixed. The data FIFO depth is 16, and QD FIFO depth is the same as data FIFO. The calculation formula of SP_ADDR_WIDTH is as Part 5. The allocation structure of SRAM is shown in the figure below:

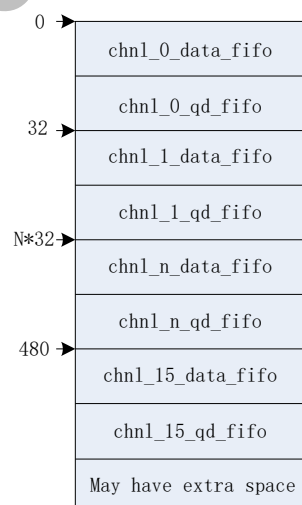


Fig. 6-7 SRAM structure

7. Register File and Memory Mapping Description

7.1. Register and Memory Mapping Summary Table

Offset Address	Name	Description
0x000	CHNL0_SRC_ADDR_L	Channel 0 Source address low register
0x004	CHNL0_SRC_ADDR_H	Channel 0 Source address high register. Only used when AHB address width>32, otherwise, it is reserved
0x008	CHNL0_DST_ADDR_L	Channel 0 Destination address low register
0x00C	CHNL0_DST_ADDR_H	Channel 0 Destination address high register. Only used when AHB address width>32, otherwise, it is reserved
0x010	CHNL0_CONFIG	Channel 0 Configuration register
0x014	CHNL0_TRANS_SIZE	Channel 0 Transfer Size register
0x018	CHNL0_QUEUE_ADDR_L	Channel 0 queue ring buffer: base address low register
0x01C	CHNL0_QUEUE_ADDR_H	Channel 0 queue ring buffer: base address high register. Only used when AHB address width>32, otherwise, it is reserved
0x020	CHNL0_QUEUE_LEN	Channel 0 queue ring buffer: length with unit byte
0x024	CHNL0_QUEUE_IDX	Channel 0 queue ring buffer: QD index of software writing and hardware reading. If the QD is 16 bytes format, then system memory address corresponding with IDX is: system memory address=16*IDX. And if the QD is 32 bytes format, then system memory address = 32*IDX
0x028	CHNL0_CONTRL	Channel 0 Control register
0x02C	CHNL0_INT_MASK	Channel 0 Interrupt mask register
0x030	CHNL0_INT_STATUS	Channel 0 Interrupt status register
0x034	CHNL0_STATUS	Channel 0 status register
0x040~0x074	CHNL1 registers	The registers of Channel 1 which have the same order as Channel 0 This range is reserved if Channel 1 does not exist.
0x080~0x0B4	CHNL2 registers	The registers of Channel 2 which have the same order as Channel 0

		This range is reserved if Channel 2 does not exist.
0x0C0~0x0F4	CHNL3 registers	The registers of Channel 3 which have the same order as Channel0 This range is reserved if Channel 3 does not exist.
0x100~0x134	CHNL4 registers	The registers of Channel 4 which have the same order as Channel0 This range is reserved if Channel 4 does not exist.
0x140~0x174	CHNL5 registers	The registers of Channel 5 which have the same order as Channel0 This range is reserved if Channel 5 does not exist.
0x180~0x1B4	CHNL6 registers	The registers of Channel 6 which have the same order as Channel0 This range is reserved if Channel 6 does not exist.
0x1C0~0x1F4	CHNL7 registers	The registers of Channel 7 which have the same order as Channel0 This range is reserved if Channel 7 does not exist.
0x200~0x234	CHNL8 registers	The registers of Channel 8 which have the same order as Channel0 This range is reserved if Channel 8 does not exist.
0x240~0x274	CHNL9 registers	The registers of Channel 9 which have the same order as Channel 0 This range is reserved if Channel 9 does not exist.
0x280~0x2B4	CHNL10 registers	The registers of Channel 10 which have the same order as Channel0 This range is reserved if Channel 10 does not exist.
0x2C0~0x2F4	CHNL11 registers	The registers of Channel 11 which have the same order as Channel0 This range is reserved if Channel 11 does not exist.
0x300~0x334	CHNL12 registers	The registers of Channel 12 which have the

		same order as Channel0 This range is reserved if Channel 12 does not exist.
0x340~0x374	CHNL13 registers	The registers of Channel 13 which have the same order as Channel0 This range is reserved if Channel 13 does not exist.
0x380~0x3B4	CHNL14 registers	The registers of Channel 14 which have the same order as Channel0 This range is reserved if Channel 14 does not exist.
0x3C0~0x3F4	CHNL15 registers	The registers of Channel 15 which have the same order as Channel0 This range is reserved if Channel 15 does not exist.
0x800	DMAC_RST	DMAC Control Register
0x804	DMAC_INT_STATUS	DMAC Interrupt Status Register
0x808	DMAC_INTMSK	DMAC Interrupt Mask Register
0x80C	DMAC_CAP	DMAC capability Register

7.2. Channel Register Description

7.2.1. CHNLn_SRC_ADDR_L

Before enabling Channel n ($n=0,1,\dots$), software should program this register with correct source address.
When transfer is in progress, software reads this register for current source starting address.

Channel n Source Address Low Register

Offset: $0x000+n*0x40$ ($n=0,1,\dots$)

Field Name	Bits	Reset Value	Direction	Description
CH_SRC_ADDR_L	31:0	32'h0	R/W	Source starting lower address of Channel n . It should be byte align. It is only used in direct DMA mode. In ring buffer descriptor mode, the source address is determined by QD.

7.2.2. CHNLn_SRC_ADDR_H

Before enabling Channel n ($n=0,1,\dots$), software should program this register with correct source address.
When transfer is in progress, software reads this register for current source starting address.

Channel n Source Address High Register

Offset: $0x004+n*0x40$ ($n=0,1,\dots$)

Field Name	Bits	Reset Value	Direction	Description
CH_SRC_ADDR_H	31:0	32'h0	R/W	Source starting higher address of Channel n .

				Only used when AHB address width>32, otherwise, it is reserved to 0. It is only used in direct DMA mode. In ring buffer descriptor mode, the source address is determined by QD.
--	--	--	--	--

7.2.3. CHNL n _DST_ADDR_L

Before enabling the channel n ($n=0,1,\dots$), software should program this register with correct destination address. When transfer is in progress, software reads this register for current destination start address.

Channel n Destination Address Low Register

Offset: $0x008+n*0x40$ ($n=0,1,\dots$)

Field Name	Bits	Reset Value	Direction	Description
CH_DST_ADDR_L	31:0	32'h0	R/W	Destination starting low address of Channel n . It should be byte align. It is only used in direct DMA mode. In ring buffer descriptor mode, the source address is determined by QD.

7.2.4. CHNL n _DST_ADDR_H

Before enabling the channel n ($n=0,1,\dots$), software should program this register with correct destination address. When transfer is in progress, software reads this register for current destination start address.

Channel n Destination Address High Register

Offset: $0x00C+n*0x40$ ($n=0,1,\dots$)

Field Name	Bits	Reset Value	Direction	Description
CH_DST_ADDR_L	31:0	32'h0	R/W	Destination starting high address of Channel n . Only used when AHB address width>32, otherwise, it is reserved to 0. It is only used in direct DMA mode. In ring buffer descriptor mode, the source address is determined by QD.

7.2.5. CHNL n _CONFIG

This register contains the configuration information of each channel. It is used in both direct DMA and ring buffer descriptor mode.

Channel n Configure Register

Offset: $0x010+n*0x40$ ($n=0,1,\dots$)

Field Name	Bits	Reset Value	Direction	Description
SRC_WIDTH	31:29	3'h0	R/W	Source transfer width setting low bits: 3'b000: 8bit (1 byte) 3'b001: 16bit (2 bytes) 3'b010: 32bit (4 bytes)

				3'b011: 64bit (8 bytes) Other: reserved
DST_WIDTH	28:26	3'h0	R/W	Destination transfer width setting low bits. 3'b000: 8bit (1 byte) 3'b001: 16bit (2 bytes) 3'b010: 32bit (4 bytes) 3'b011: 64bit (8 bytes) Other: reserved
SRC_FIXED	25	1'b0	R/W	Source address increment or not. When set to 1, the source address is fixed after each transfer. Note: When the data of the peripheral is stored in FIFO, it means that the access port is fixed. At this time, it is recommended to set this bit to 1 and the transmission type to Single (SRC_BST_LEN=0).
DST_FIXED	24	1'b0	R/W	Destination address increment or not. When set to 1, the destination address is fixed after each transfer. Note: When the data of the peripheral is stored in FIFO, it means that the access port is fixed. At this time, it is recommended to set this bit to 1 and the transmission type to Single (DST_BST_LEN=0).
SRC_BST_LEN	23:20	4'h0	R/W	Source burst length. AHB burst length is not greater than SRC_BST_LEN. The value mainly depends on peripheral using DMA_REQ/DMA_ACK as hardware handshake. For example, if DMA_REQ/DMA_ACK pair require 4 beat data, then SRC_BST_LEN=3. General using for peripheral, SRC_BST_LEN=0. But for memory using, it should be 4'hf for performance concern. To align with AXIDMAC, SRC_BST_LEN is recommended to 0/3/7/15. For others, it is reserved.
DST_BST_LEN	19:16	4'h0	R/W	Destination burst length. The same description as SRC_BST_LEN.
SRC_HPROT	15:12	4'h0	R/W	Source hprot signal

DST_HPROT	11:8	4'h0	R/W	Destination hprot signal. Note: For the last write transaction, hprot [2] will be overwritten to 1, this means that the last write transaction is non-bufferable.
Reserve	7:0	8'h0	R/W	Reserve

7.2.6. CHNLn_TRANS_SIZE

This register contains the transfer size of each channel. It is used only in both direct DMA mode.

Channel *n* Transfer Size Register

Offset: $0x014+n*0x40$ ($n=0,1,...$)

Field Name	Bits	Reset Value	Direction	Description
TRANS_SIZE	31:0	32'h0	R/W	Transfer size in unit byte.

7.2.7. CHNLn_QUEUE_ADDR_L

This register contains the lower base address, pointing to the queue ring buffer. It is only used in ring buffer descriptor mode.

Channel *n* Queue Ring Buffer Low Base Address

Offset: $0x018+n*0x40$ ($n=0,1,...$)

Field Name	Bits	Reset Value	Direction	Description
CH_QUEUE_ADDR_L	31:0	32'h0	R/W	Queue ring buffer base lower address. It must be aligned with QDs size. For example, if the QD is 16 bytes format, and the queue ring buffer can store 16 QDs, then QDs whole size = 256byte. So CH_QUEUE_ADDR_L[7:0] = 8'h0.

7.2.8. CHNLn_QUEUE_ADDR_H

This register contains the higher base address, pointing to the queue ring buffer. It is only used in ring buffer descriptor mode and the AHB bus address width is > 32 bit.

Channel *n* Queue Ring Buffer High Base Address

Offset: $0x01C+n*0x40$ ($n=0,1,...$)

Field Name	Bits	Reset Value	Direction	Description
CH_QUEUE_ADDR_H	31:0	32'h0	R/W	Queue ring buffer base higher address.

7.2.9. CHNLn_QUEUE_LEN

This register contains the number of QD in queue ring buffer. The ring buffer is allocated to store $2*N$ ($N \geq 1$) QD. It is only used in ring buffer descriptor mode.

Channel *n* Queue Ring Buffer High Base Address

Offset: $0x020+n*0x40$ ($n=0,1,...$)

Field Name	Bits	Reset Value	Direction	Description
Reserved	31:16	16'h0	R/W	Reserved

CH_QUEUE_LEN	15:0	16'h0	R/W	The QD numbers which queue ring buffer can contains in system memory.
--------------	------	-------	-----	---

7.2.10. CHNLn_QUEUE_IDX

This register contains the index of the next QD software writing and hardware reading. It is only used in ring buffer descriptor mode.

Channel n Queue Ring Buffer Software Write QD Index Address Offset: $0x024+n*0x40$ ($n=0,1,\dots$)

Field Name	Bits	Reset Value	Direction	Description
CH_QUEUE_SW_WR_IDX	31:16	16'h0	R/W	Index of the next QD in the queue ring buffer software is writing. The index width is $\log_2(\text{CH_QUEUE_LEN})+1$. For example, if $\text{CH_QUEUE_LEN}=16$, then the queue ring buffer can store at most 16 QDs. The index width is $\log_2(16)+1=5$. So the index software writing is repeat 0~31. The highest bit of index is used for ring buffer empty/full determination. If $\text{SW_WR_IDX}=\text{HW_RD_IDX}$, then buffer is empty, if $\text{SW_WR_IDX}-\text{HW_RD_IDX}=16$, then the buffer is full.
CH_QUEUE_HW_RD_IDX	15:0	16'h0	RU	Index of the next QD in the queue ring buffer hardware reading. The same description as CH_QUEUE_SW_WR_IDX.

7.2.11. CHNLn_CONTROL

This register contains the controlling information which is necessary for a channel to run correctly and definitely. It is used in both direct DMA and ring buffer descriptor mode.

Channel n Control Register Offset: $0x028+n*0x40$ ($n=0,1,\dots$)

Field Name	Bits	Reset Value	Direction	Description
Reserve	31:23	9'h0	R/W	Reserve
DMA_MODE	22	1'b0	R/W	DMA mode selection. It is static configure before starting DMA transaction 1'b0: Ring buffer descriptor mode 1'b1: Direct DMA mode
SRC_HSK_EN	21	1'b0	R/W	Source handshake enable 1'b0: Source transfer does not consider handshake, if the source is memory, it must be 0 1'b1: Source transfer considers handshake, it is usually used in peripheral depending on

				peripheral support request/handshake or not
DST_HSK_EN	20	1'b0	R/W	Destination handshake enable 1'b0: Destination transfer does not consider handshake, if the destination is memory, it must be 0 1'b1: Destination transfer considers handshake, it is usually used in peripheral depending on peripheral support request/handshake or not
SRC_FLOW_CTL	19	1'b0	R/W	Source software/hardware control flow selection 1'b0: Software control flow 1'b1: Hardware control flow Note: no support that SRC&DST both hardware control.
DST_FLOW_CTL	18	1'b0	R/W	Destination software/hardware control flow selection 1'b0: Software control flow 1'b1: Hardware control flow Note: no support that SRC&DST both hardware control.
Reserve	17	1'b0	R/W	Reserve
SRC_PERI	16:13	4'b0	R/W	Select DMA handshake for the source peripheral. Only effective SRC_HSK_EN = 1, otherwise ignored.
Reserve	12	1'b0	R/W	Reserve
DST_PERI	11:8	4'b0	R/W	Select DMA handshake for the destination peripheral. Only effective DST_HSK_EN = 1, otherwise ignored.
Reserve	7:3	5'b0	R/W	Reserve
CH_ABORT	2	1'b0	R/W	Abort the channel. The DMA channel is reset except dmac_reg after CH_ABORT is asserted. SW can sets this bit to abort the channel. All the HW logic return to idle and the internal FIFO is flushed. After the abort is completed, interrupt is generated. SW can re-config the dma to start the new transfer.
Reserve	1	1'b0	R/W	Reserve

CH_EN	0	1'b0	R/W	Channel enable. 1'b0: disable a channel 1'b1: enable a channel Clearing the Enable bit by software causes the current AHB burst, if one is in progress, to complete, and the channel is then disabled. The difference between disable and abort is: the channel will be reset after abort.
-------	---	------	-----	---

7.2.12. CHNL_n_INT_MASK

This register is used to mask the Channel interrupt status register.

Channel *n* Interrupt Mask Register

Offset: 0x02C+n*0x40 (n=0,1,...)

Field Name	Bits	Reset Value	Direction	Description
Reserved	31:6	26'h0	RW	Reserved
BUSERR_MASK	5	1'b0	RW	BUSERR_INT mask 1'b0: unmask the interrupt source 1'b1: mask the interrupt source
QD_IOC_MASK	4	1'b0	RW	QD_IOC_INT mask 1'b0: unmask the interrupt source 1'b1: mask the interrupt source
QUEUE_SUSP_MASK	3	1'b0	RW	QUEUE_SUSP_INT mask 1'b0: unmask the interrupt source 1'b1: mask the interrupt source
ABORTDN_MASK	2	1'b0	RW	ABORTDN_INT mask 1'b0: unmask the interrupt source 1'b1: mask the interrupt source
DISABLEDN_MASK	1	1'b0	RW	DISABLEDN_INT mask 1'b0: unmask the interrupt source 1'b1: mask the interrupt source
TRANSDN_MASK	0	1'b0	RW	TRANSDN_INT mask 1'b0: unmask the interrupt source 1'b1: mask the interrupt source

7.2.13. CHNL_n_INT_STATUS

This register shows operation status of each channel.

Channel *n* Interrupt Status Register

Offset: 0x030+n*0x40 (n=0,1,...)

Field Name	Bits	Reset Value	Direction	Description
Reserve	31:6	26'h0	R/W	Reserved.
BUSERR_STS	5	1'b0	W1C	BUSERR_INT raw status, high active This error will cancel the whole block transfer. Writing "1" to clear.
QD_IOC_STS	4	1'b0	W1C	QD_IOC_INT raw status, high active. It is only used in ring buffer descriptor mode, when the IOC field of QD is set, then after the QD is completed, interrupt is triggered. Writing "1" to clear.
QUEUE_SUSP_STS	3	1'b0	W1C	QUEUE_SUSP_INT raw status, high active. It is only used in ring buffer descriptor mode, when the queue ring buffer is empty, interrupt is triggered to inform CPU preparing new QDs. Writing "1" to clear.
ABORTDN_STS	2	1'b0	W1C	ABORTDN_INT(abort done) raw status, high active. Please refer to CHNLn_CONTROL.CH_ABORT description. Writing "1" to clear.
DISABLEDN_STS	1	1'b0	W1C	DISABLEDN_INT(disable done) raw status, high active. Please refer to CHNLn_CONTROL.CH_EN description. Writing "1" to clear.
TRANSDN_STS	0	1'b0	W1C	TRANSDN_INT(transfer done, all the transfer size is completed) raw status, high active. Generally used in direct DMA mode, when transfer size data is completed, interrupt is triggered. Writing "1" to clear.

7.2.14. CHNLn_STATUS

This register is status register of each channel.

Channel *n* Status Register

Offset: $0x034+n*0x40$ ($n=0,1,\dots$)

Field Name	Bits	Reset Value	Direction	Description
REMAIN_TRANS_SIZE	31:0	32'h0	RU	Remain transfer size. Generally it is used in channel disable. After the SW disable the channel or the channel hang. Then SW can read this register to know how much data

				transaction is completed.
--	--	--	--	---------------------------

7.3. Control Register Description

7.3.1. DMAC_RST

This register contains the control bit of VS_AHBDMAC.

DMAC Control Register

Offset: 0x800

Field Name	Bits	Reset Value	Direction	Description
Reserve	31:1	31'h0	R/W	Reserved.
SOFT_RST	0	0x0	R/W	soft reset signal and reset whole except RW register. It will active immediately causing the bus hang risk, and when the reset assert, the dmac chnl_en will disable. Software should clear this bit to 0 later. Note: If want to reset dmac without risk, SW can used chnl_abort for all chnl.

7.3.2. DMAC_INT_STATUS

This register shows the masked interrupt status from each channel, including all active interrupts of that channel.

DMAC Interrupt Register

Offset: 0x804

Field Name	Bits	Reset Value	Direction	Description
Reserve	31:16	16'h0	RW	Reserve
CH15_INT	15	1'b0	RU	Interrupt status of channel 15 after masking. High active. This bit is reserved if Channel 15 does not exist.
...	...	...	...	...
CH3_INT	3	1'b0	RU	Interrupt status of channel 3 after masking. High active. This range is reserved if Channel 3 does not exist.
CH2_INT	2	1'b0	RU	Interrupt status of channel 2 after masking. High active. This range is reserved if Channel 2 does not exist.

CH1_INT	1	1'b0	RU	Interrupt status of channel 1 after masking. High active. This range is reserved if Channel 1 does not exist.
CH0_INT	0	1'b0	RU	Interrupt status of channel 0 after masking. High active.

7.3.3. DMAC_INTMSK

This register shows the mask bit per channel, merging all interrupt types together, only mask dmac_int and not effect status register.

DMAC Interrupt Mask Register

Offset: 0x808

Field Name	Bits	Reset Value	Direction	Description
Reserve	31:16	16'h0	RW	Reserve
CH15_INT_MASK	15	1'b1	RW	Interrupt mask of channel 15. Value "1" on this bit will mask CH15_INT. This bit is not used if Channel 15 does not exist.
...	...	...	...	...
CH3_INT_MASK	3	1'b1	RW	Interrupt mask of channel 3. Value "1" on this bit will mask CH3_INT. This bit is not used if Channel 3 does not exist.
CH2_INT_MASK	2	1'b1	RW	Interrupt mask of channel 2. Value "1" on this bit will mask CH2_INT. This bit is not used if Channel 2 does not exist.
CH1_INT_MASK	1	1'b1	RW	Interrupt mask of channel 1. Value "1" on this bit will mask CH1_INT. This bit is not used if Channel 1 does not exist.
CH0_INT_MASK	0	1'b1	RW	Interrupt mask of channel 0. Value "1" on this bit will mask CH0_INT.

7.3.4. DMAC_CAP

This register shows the capability of DMAC

DMAC Capability Register

offset: 0x80C

Field Name	Bits	Reset Value	Direction	Description
------------	------	-------------	-----------	-------------

CHN_NUM	31:26	NA	RO	Channel number
DATA_WIDTH	25:24	NA	RO	Bus data width 2'b00: 32 bit 2'b01: 64 bit Others: Reserve
ADDR_WIDTH	23:17	NA	RO	Bus address width
AMBA_PROTOCOL	16	NA	RO	AMBA protocol 1'b0: AXI 1'b1: AHB
Reserve	15:0	16'h0	RW	Reserve

8. Interface Timing Diagram

8.1. Peripheral Handshaking Interface

Some peripheral need dma_req/dma_ack handshake working with DMAC. For memory-to-peripheral, if the peripheral is ready to accept, it asserts dma_req to DMAC, then DMAC send burst data to peripheral and assert dma_ack. After peripheral de-assert dma_req, dma_ack is de-asserted. And for peripheral-to-memory, if the peripheral is ready to transmit, it asserts dma_req to DMAC, then DMAC send read command to the peripheral. After the data is read from peripheral, dma_ack asserted by DMAC until dma_req is de-asserted.

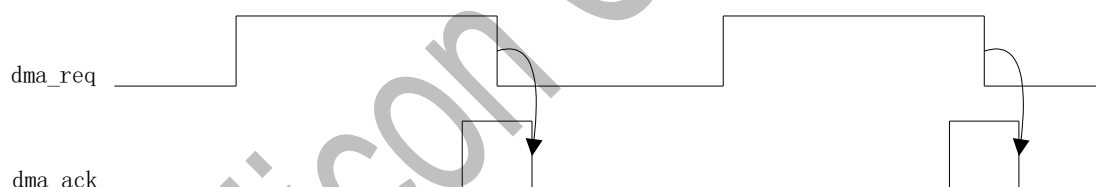
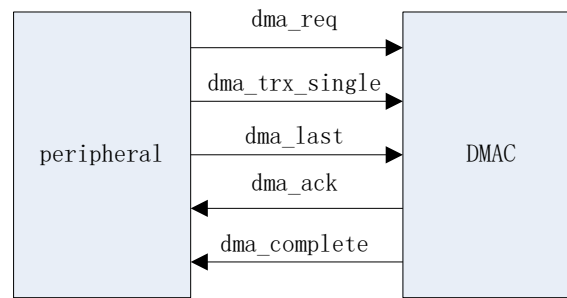
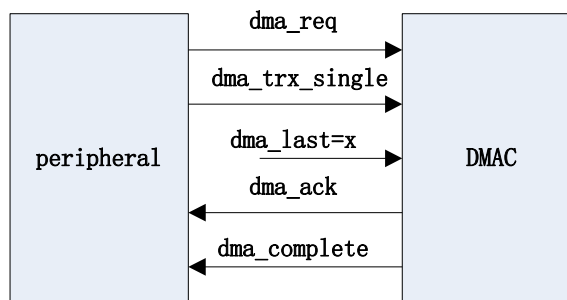


Fig 8-1 DMA channel handshake

There are two control flows for DMA transfer: DMAC control flow and peripheral control flow. In DMAC control flow, all the transfer data block size is determined by CHNLn_TRANS_SIZE in direct DMA mode or QD.transfer_data_length in ring buffer mode. Dma_trx_single is sampled by the DMAC only in the single transaction region of the block transfer, dma_last is ignored in DAMC control flow. In address aligned case, if dma_req is asserted, a burst data is read from source or written to destination until the last remain data is less than one burst. For the last remain data less than one burst, the DMA determine the burst type. In peripheral control flow, the actual data transfer block size is determined by peripheral, In address aligned case, if dma_req = 1 and dma_trx_single=0, then a burst data is transferred. If dma_req = 1 and dma_trx_single=1, then a single data is transferred. If dma_req=1 and dma_last=1, means the peripheral does not have data any more, burst or single is depended on dma_trx_single. In peripheral control flow, it is recommended that the transfer size is set a large value, such as 32'hffffff. After the last data is completed, dma_complete assert, and interrupt is triggered or QD is expired, even if the remain transfer size does not decrease to 0.



Hardware control flow



Software control flow

8.2. APB Bus Interface

For more details please refer to the AMBA4 APB specification.

8.3. AHB Bus Interface

For more details, please refer to the AMBA3 AHB Specification.

9. Configuration Program Flow

Programming flow for Direct Mode Transfer:

1. Software reads the DMAC channel enable to select the available channel.
2. Software programs the CHNL_SRC/DST_ADDR register, CHNL_TRANS_SIZE register and CHNL_CONFIG register with appropriate values for transfer.
3. Software programs the CHNL_CONTROL register for handshake operation and CHNL_CONTROL.DMA_MODE register must be set to 1'b1.
4. Software enables the channel by writing 1 to CHNL_CONTROL.CH_EN register.
5. DMAC initiates the block transfer based on the settings. After the block size completed, the interrupt will be generated.

Programming flow for Ring buffer Mode Transfer:

1. Software reads the DMAC channel enable to select the available channel.
2. Software programs the CHNL_QUEUE_ADDR_L/H register and CHNL_CONFIG register with appropriate values for transfer.
3. Software programs the CHNL_CONTROL register for handshake operation and

CHNL_CONTROL.DMA_MODE register must be set to 1'b0.

4. Software programs the base address of the first ringbuffer descriptor in CHNL_QUEUE_ADDR register and the number of QD in CHNL_QUEUE_LEN register.

5. Software create one or more QD in system memory, and programs the available QD number in CHNL_QUEUE_IDX register.

6. Software enables the channel by writing 1 to CHNL_CONTROL.CH_EN.

7. DMAC initiates the block transfer based on the settings. After there is no QD available, the interrupt will be generated.

10. Application Note

10.1. Gate Count

Lib: TSMC_SNPS05A_210H_POK_ULVTLL_CMIN_TT_UV_NT

NAND Gate Unit Name: HDBSVT06_ND2_2

NAND Gate Unit Area: 0.05355um²

The parameter settings are as follows:

DATA_WIDTH=32; (AHB interface data bit width)

ADDR_WIDTH=32; (AHB interface address bit width)

CHNL_NUM=16; (Number of channels)

HS_NUM=16; (Number of DMA handshake interfaces)

The target frequency for both PCLK and HCLK is 100MHz

The total gate count is 150 thousand

10.2. Channel Number Definition

10.3. System Considerations

Reducing the number of transactions that occur on the buses reduces the latency on the bus, improves system performance, and reduces power consumption. Therefore, the following design considerations are recommended:

- All memory transactions are, in the standard configuration, DATA_WIDTH bits wide to improve bus efficiency.
- Slow peripherals that normally use wait state must contain FIFOs so that data can be transferred at full speed using burst transfers.

10.4. Other Information

11. Absolute MAHBmum Ratings

Table 1: Power supply mAHBmum ratings

Parameter	Symbol	Unit	Min.	Max.	Notes

12. Recommended Operation Conditions

Table 2: Recommended operating conditions

Parameter	Symbol	Unit	Min.	Max.	Notes

13. Electrical Characteristics

Table 3. Electrical characteristics

Parameters	Symbol	Conditions	Min.	Typ.	Max.	Unit
DC Characteristics (<i>conditions</i>)						
AC Characteristics (<i>conditions</i>)						
Others (<i>conditions</i>)						

Notes:

(Conditions or test settings for electrical characteristics definitions)

14. User Guide

14.1. Integrating with the Application RTL

vsi_ahbdmac is the top module used in our designs. user only need to have a sample instantiation of vsi_ahbdmac to Integrating with the Application RTL as follows:

```
vsi_ahbdmac #(
    ADDR_WIDTH(32),
    DATA_WIDTH(64),
    CHNL_NUM(16),
    HS_NUM(16)
)
u_vsi_ahbdmac (
    /*input*/
    /*output*/
);
```

Parameter		
ADDR_WIDTH	NA	AHB master address width
DATA_WIDTH	NA	AHB master data width, 32bit or 64bit
CHN_NUM	NA	DMA channel number, from 1 to 16
HS_NUM	NA	DMA handshake number, from 1 to 16

NOTE: the /*input*/ and /*output*/ refer to 5 section (Pin List and Assignment)

14.2. Integrating with single port SRAM

The calculation formula of local parameter SP_ADDR_WIDTH is as follows:

$$2^{SP_ADDR_WIDTH} = 32 \times 2^{\lceil \log_2 CHNL_NUM \rceil}$$

The number 32 is come from 16+16, because the FIFO depth for storing data in each channel is 16, and the FIFO depth for storing QD in each channel is also 16. “ $\lceil \cdot \rceil$ ” is the round-up function. for example, if the CHNL_NUM is 12, the SP_ADDR_WIDTH is equal to 9. Suppose the size of SRAM is DEPTH x WIDTH, the WIDTH is equal to DATA_WIDTH, and the DEPTH should be greater than or equal to 32 X CHNL_NUM, It depends on whether the Memory Compiler can generate SRAM with the right depth.